The AutoTraQ backend is the “server side” of the project. It’s a REST API built with Express.js and TypeScript, and it uses Prisma to talk to a MySQL database. The backend is organized in a clean, layered way: a request comes in through a route, middleware runs first to check things like login tokens and input rules, then a controller receives the request, a service applies the business logic, and Prisma runs the database queries. This structure helps keep the code easier to read, test, and maintain because each layer has a clear job.

In the codebase, index.ts starts the Express server. The routes folder connects URLs (endpoints) to the correct controller functions. Controllers are basically the “traffic cops” for requests—they read the request, pull out the needed data, and call the right service. Services are where the real rules of the app live (for example, how to create a part, approve a request, or record inventory changes). Middleware runs before controllers and is used for things like verifying a user is logged in with a JWT token, checking user roles (admin, manager, fulfillment, viewer), handling errors, and validating request data using Zod so bad or missing data doesn’t reach the database. Prisma is the tool that actually reads/writes data in MySQL, and its schema file defines the tables and relationships.

The backend supports key features through endpoints like logging in (/api/auth/login) and registering (/api/auth/register), managing parts (/api/parts), tracking inventory events (/api/inventory), listing vehicles (/api/vehicles), handling part requests (/api/requests), viewing the audit trail (/api/audit), and doing CSV bulk import (/api/csv/import). The login system works by taking an email and password, hashing and comparing the password using bcrypt, then returning a signed JWT that the frontend sends on future requests in the Authorization: Bearer <token> header. Protected routes check that token in the auth middleware and also enforce role-based permissions.

To run it locally, you go into the backend folder, install dependencies, generate the Prisma client, push the schema to the database, and start the dev server (usually on http://localhost:3002). Testing is done with Vitest, with both unit tests (testing small pieces) and integration tests (testing bigger workflows end-to-end, like request-to-fulfillment). Overall, the main meeting talking points are that the backend uses a layered architecture for clarity, Prisma for type-safe database access, JWT + bcrypt for standard authentication, Zod for strong input validation, Vitest for testing, an audit trail for accountability, and role-based access control for security and proper permissions.